

SUBJECT NAME:	COMPUTER ORGANIZATION AND ARCHITECTURE				
SUBJECT CODE:	SECR 1033				
SEMESTER:	2 – 2023/2024				
LAB TITLE:	Lab 2: Arithmetic Equations & Operations				
STUDENT INFO :	Execute the lab in group of two. <table> <tr> <th>Student 1</th> <th>Student 2</th> </tr> <tr> <td>No. 1, 3, 5</td> <td>No 2, 4, 6</td> </tr> </table> Name 1: <u>Lau Yee Wen</u> Metric No: <u>A23CS0099</u> Name 2: <u>Cheryl Cheong Kah Voon</u> Metric No: <u>A23CS0060</u>	Student 1	Student 2	No. 1, 3, 5	No 2, 4, 6
Student 1	Student 2				
No. 1, 3, 5	No 2, 4, 6				
SUBMISSION DATE:	<u>-5-2024</u>				

MARKS:

Arithmetic Equation Coding in Assembly Language

- Q1. Execute the program below. Determine output of the program by inspecting the content of the related registers.
- a) Fill in Table 1 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.
- b) Paste the screenshot of all registers' content after each LINE is executed.

```
INCLUDE Irvine32.inc
.data
var1 word 1
var2 word 9

.code
main PROC
    mov ax, var1      ; LINE1
    mov bx, var2      ; LINE2
    xchg ax, bx       ; LINE3
    mov var1, ax      ; LINE4
    mov var2, bx      ; LINE5
    call DumpRegs
    exit
main ENDP
END main
```

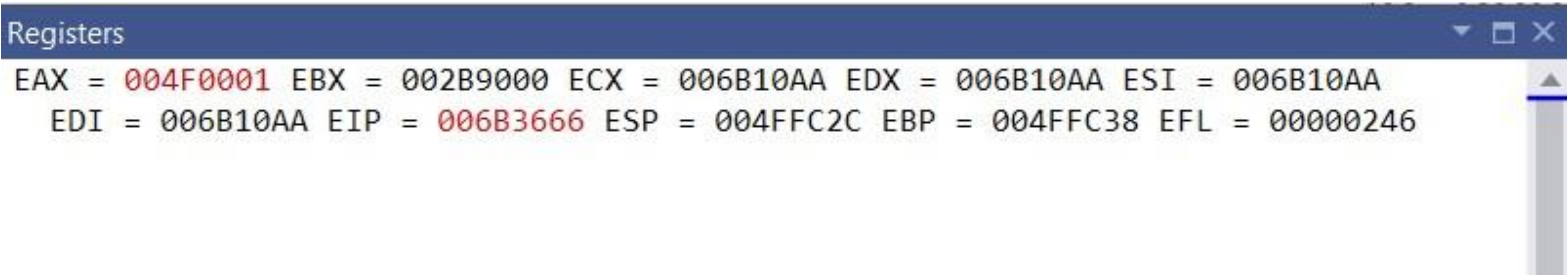
Answer Q1

- a) Fill (Write) in the contents for the related register in each line:

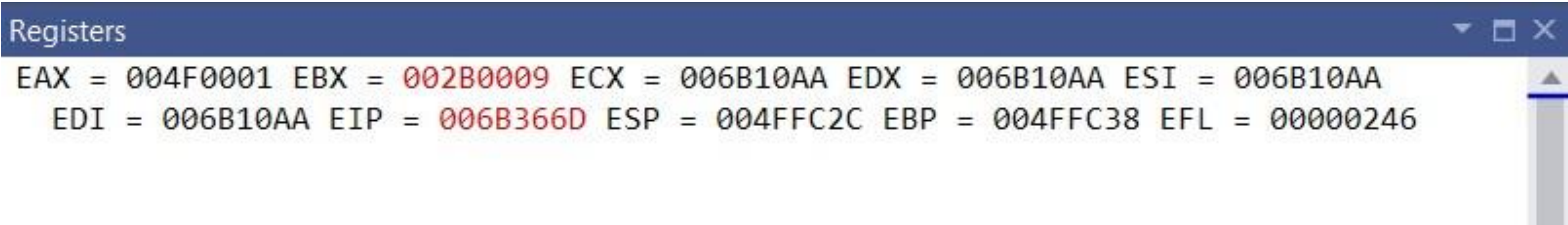
Table 1		
LINE1	AX = 0001h var1 = 0001h	Move the value of var1 (1d) into register AX
LINE2	BX = 0009h var2 = 0009h	Move the value of var2 (9d) into register BX
LINE3	AX = 0009h BX = 0001h	Exchange the value of register AX and BX
LINE4	AX = 0009h var1 = 0009h	Move the value of register AX into variable var1
LINE5	BX = 0001h var2 = 0001h	Move the value of register BX into variable var2

- b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:



LINE2:



LINE3:

Registers

EAX = 004F0009 EBX = 002B0001 ECX = 006B10AA EDX = 006B10AA ESI = 006B10AA
EDI = 006B10AA EIP = 006B366F ESP = 004FFC2C EBP = 004FFC38 EFL = 00000246

LINE4:

Registers

EAX = 004F0009 EBX = 002B0001 ECX = 006B10AA EDX = 006B10AA ESI = 006B10AA
EDI = 006B10AA EIP = 006B3675 ESP = 004FFC2C EBP = 004FFC38 EFL = 00000246

Autos

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
ax	0x0009	unsigned short
bx	0x0001	unsigned short
var1	0x0009	unsigned short
var2	0x0009	unsigned short

LINE5:

Registers

EAX = 004F0009 EBX = 002B0001 ECX = 006B10AA EDX = 006B10AA ESI = 006B10AA
EDI = 006B10AA EIP = 006B367C ESP = 004FFC2C EBP = 004FFC38 EFL = 00000246

Autos

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
ax	0x0009	unsigned short
bx	0x0001	unsigned short
var1	0x0009	unsigned short
var2	0x0001	unsigned short

Q2. Execute the program below. Determine output of the program by inspecting the content of the related registers and watches.

a) Fill in Table 2 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $Rval = (-Xval + (Yval - Zval)) + 1$

```
include Irvine32.inc

.data
Rval DWORD ?
Xval DWORD 26
Yval DWORD 30
Zval DWORD 40

.code
main proc
    mov eax,Xval      ; LINE1
    neg eax           ; LINE2
    mov ebx,Yval      ; LINE3
    sub ebx,Zval      ; LINE4
    add eax,ebx       ; LINE5
    inc eax           ; LINE6
    mov Rval,eax      ; LINE7
    exit
main endp
end main
```

Answer Q2

a) Fill (Write) in the contents for the related register in each line:

Table 2

LINE1	EAX = 0000001Ah Xval = 0000001Ah	Move the value of Xval (26d) into register EAX
LINE2	EAX = FFFFFFF6h	Negate register EAX with 2's complement
LINE3	EBX = 0000001Eh Yval = 0000001Eh	Move the value of Yval into register EBX
LINE4	EBX = FFFFFFF6h Zval = 00000028h	Subtract the value of register EBX with Zval.
LINE5	EAX = FFFFFFFCh EBX = FFFFFFF6h	Add value of register EBX to the register EAX
LINE6	EAX = FFFFFFFDh	Increment EAX by 1
LINE7	EAX = FFFFFFFDh Rval = FFFFFFFDh	Store the value of register EAX into Rval

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

Registers

EAX = 0000001A EBX = 01009000 ECX = 00031005 EDX = 00031005
ESI = 00031005 EDI = 00031005 EIP = 00031015 ESP = 00FCF9D4
EBP = 00FCF9E0 EFL = 00000246

LINE2:

Registers

EAX = FFFFFFFE6 EBX = 00DAC000 ECX = 00031005 EDX = 00031005
ESI = 00031005 EDI = 00031005 EIP = 00031017 ESP = 00F8FC58
EBP = 00F8FC64 EFL = 00000293

0x00034008 = 0000001E

LINE3:

Registers

EAX = FFFFFFFE6 EBX = 0000001E ECX = 00211005 EDX = 00211005 ESI = 00211005
EDI = 00211005 EIP = 0021101D ESP = 00D3FF48 EBP = 00D3FF54 EFL = 00000293

0x0021400C = 00000028

LINE4:

Registers

EAX = FFFFFFFE6 EBX = FFFFFFFF6 ECX = 00031005 EDX = 00031005
ESI = 00031005 EDI = 00031005 EIP = 00031023 ESP = 00F8FC58
EBP = 00F8FC64 EFL = 00000287

LINE5:

Registers

EAX = FFFFFFFDC EBX = FFFFFFFF6 ECX = 00031005 EDX = 00031005
ESI = 00031005 EDI = 00031005 EIP = 00031025 ESP = 00F8FC58
EBP = 00F8FC64 EFL = 00000283

LINE6:

Registers

EAX = FFFFFFFD EBX = FFFFFFF6 ECX = 00031005 EDX = 00031005
ESI = 00031005 EDI = 00031005 EIP = 00031026 ESP = 00F8FC58
EBP = 00F8FC64 EFL = 00000287

0x00034000 = 00000000

LINE7:

Registers

EAX = FFFFFFFD EBX = FFFFFFF6 ECX = 00C21005 EDX = 00C21005 ESI = 00C21005
EDI = 00C21005 EIP = 00C2102B ESP = 00B3FDCC EBP = 00B3FDD8 EFL = 00000287

Autos

Search (Ctrl+E)

Search Depth: 3

Name	Value	Type
Rval	0xffffffff	unsigned...
eax	0xffffffff	unsigned...
ebx	0xffffffff6	unsigned...

108 %

Registers Call Stack Breakpoints Except

- Q3. Execute the program below. Determine output of the program by inspecting the content of the related registers.
- a) Fill in Table 3 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.
 - b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $var4 = [(var1 * var2) + var3] - 1$

```
include irvine32.inc

.data
var1 DWORD 5
var2 DWORD 10
var3 DWORD 20
var4 DWORD ?

.code
main proc
    mov eax, var1      ; LINE1
    mul var2           ; LINE2
    add eax, var3      ; LINE3
    dec eax            ; LINE4
    exit
main endp
end main
```

Answer Q3

- a) Fill (Write) in the contents for the related register in each line:
- Table 3

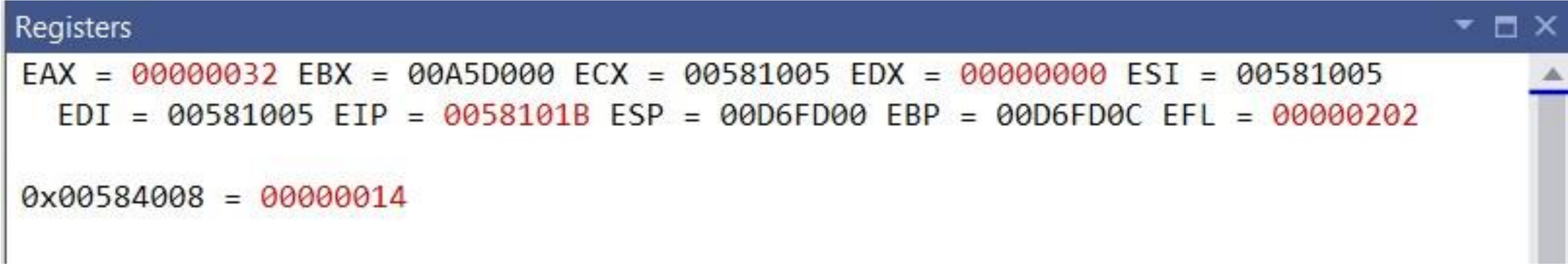
LINE1	EAX = 00000005h var1 = 00000005h	Move the value of var1 (5d) into register EAX
LINE2	EAX = 0000 0032h var2 = 0000 000Ah	Multiply the value of var2 (10d) into register EAX.
LINE3	EAX = 0000 0046h var3 = 0000 0014h	Add the value of var3 (odd) into register EAX.
LINE4	EAX = 0000 0045h var4 = 0000 0045h	Subtract 1 from the value of register and move EAX into variable var4.

b) Paste here screenshot of all registers' content after each LINE is executed:

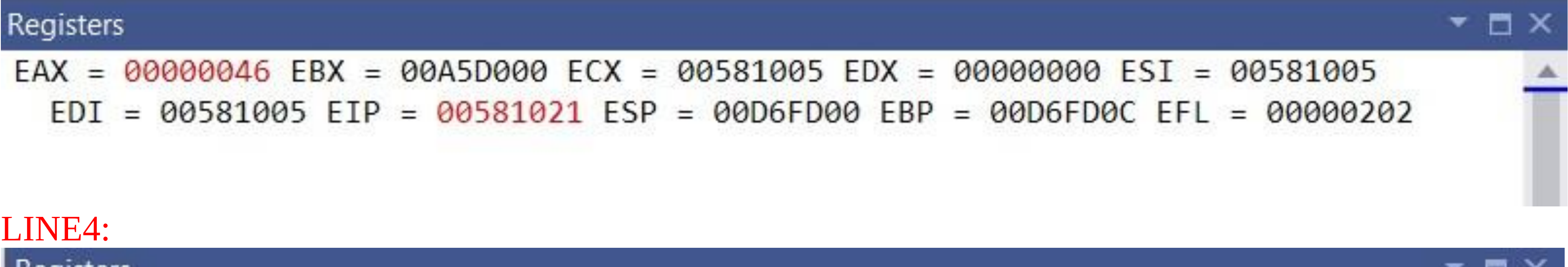
LINE1:



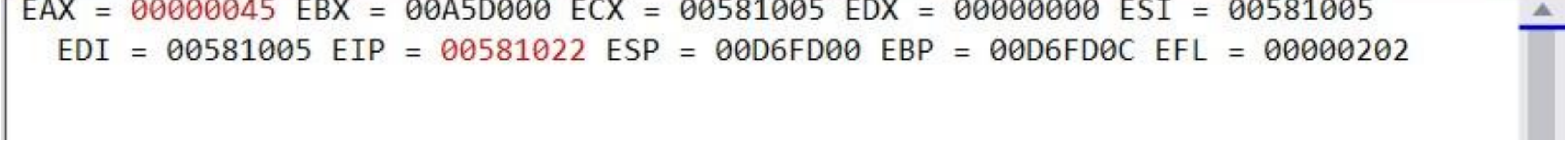
LINE2:



LINE3:



LINE4:



Q4. Execute the program below. Determine output of the program by inspecting the content of the related registers.

a) Fill in Table 4 with the content of each register or variable on every LINE, in Hexadecimal (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $\text{var4} = (\text{var1} * 5) / (\text{var2} - 3)$

```
include irvine32.inc
.data
    var1 WORD 40
    var2 WORD 10
    var4 WORD ?
.code
main proc
    mov ax,var1      ; LINE1
    mov bx,5         ; LINE2
    mul bx           ; LINE3
    mov bx,var2      ; LINE4
    sub bx,3         ; LINE5
    div bx           ; LINE6
    mov var4,ax      ; LINE7
    exit
main endp
end main
```

Answer Q4

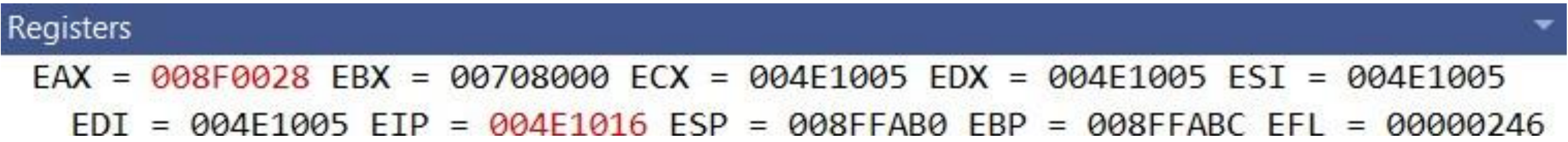
a) Fill (Write) in the contents for the related register in each line:

Table 4

LINE1	AX = 0028h var1 = 0028h	Move the value of var1 (40d) into register AX
LINE2	BX = 0005h	Move the value 5 into register BX
LINE3	AX = 00C8h BX = 0005h	Multiply the value of register BX with AX and store the value in the register DX:AX
LINE4	BX = 000Ah var2 = 000Ah	Move the value of var2 into the register of BX
LINE5	BX = 0007h	Subtract the value of register BX with and immediate value of 3
LINE6	AX = 001Ch BX = 0007h DX = 0004h	Divide the value of register AX by BX, the quotient is store in AX and the remainder store in DX
LINE7	AX = 001Ch var4 = 001Ch	Move the value of register AX into var4.

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:



LINE2:

Registers

EAX = 008F0028 EBX = 00700005 ECX = 004E1005 EDX = 004E1005 ESI = 004E1005
EDI = 004E1005 EIP = 004E101A ESP = 008FFAB0 EBP = 008FFABC EFL = 00000246

LINE3:

Registers

EAX = 008F00C8 EBX = 00700005 ECX = 004E1005 EDX = 004E0000 ESI = 004E1005
EDI = 004E1005 EIP = 004E101D ESP = 008FFAB0 EBP = 008FFABC EFL = 00000202

LINE4:

Registers

EAX = 008F00C8 EBX = 0070000A ECX = 004E1005 EDX = 004E0000 ESI = 004E1005
EDI = 004E1005 EIP = 004E1024 ESP = 008FFAB0 EBP = 008FFABC EFL = 00000202

LINE5:

Registers

EAX = 008F00C8 EBX = 00700007 ECX = 004E1005 EDX = 004E0000 ESI = 004E1005
EDI = 004E1005 EIP = 004E1028 ESP = 008FFAB0 EBP = 008FFABC EFL = 00000202

LINE6:

Registers

EAX = 008F001C EBX = 00700007 ECX = 004E1005 EDX = 004E0004 ESI = 004E1005
EDI = 004E1005 EIP = 004E102B ESP = 008FFAB0 EBP = 008FFABC EFL = 00000202

LINE7:

Registers

EAX = 008F001C EBX = 00700007 ECX = 004E1005 EDX = 004E0004 ESI = 004E1005
EDI = 004E1005 EIP = 004E1031 ESP = 008FFAB0 EBP = 008FFABC EFL = 00000202

Short Notes for MUL CX and DIV BL:

MUL CX

- a. MUL always uses AX (or its extended versions EAX or RAX) as the implicit destination register.

- b. The operand size determines the size of the result:
 - i. Byte-sized operand: Result in AX
 - ii. Word-sized operand: Result in DX:AX
 - iii. Doubleword-sized operand (32-bit mode): Result in EDX:EAX
 - iv. Quadword-sized operand (64-bit mode): Result in RDX:RAX
- c. The upper half of the result (DX or EDX or RDX) holds any overflow bits.
- d. The Carry Flag (CF) is set if the upper half of the product is non-zero.

DIV BL

- a. DIV always uses the DX:AX or EDX:EAX pair as the implicit dividend register.
- b. The divisor is specified as the operand of the DIV instruction.
- c. The quotient is stored in AX (for 16-bit division) or EAX (for 32-bit division).
- d. The remainder is stored in DX.
- e. Clear DX (or EDX for 32-bit division) before division to ensure a correct 16-bit or 32-bit dividend.
- f. If the divisor is 0, a division error occurs.
- g. The Overflow Flag (OF) is set if the quotient is too large to fit in the destination register.

Q5. Given the following instructions as is Code Snippet 1.

- a) Write a full program to execute the Code Snippet 1.
- b) What are the contents of the related registers after Code Snippet 1 is executed? Paste the screenshot of DumpReg.

; Code Snippet 1 (MUL CX)

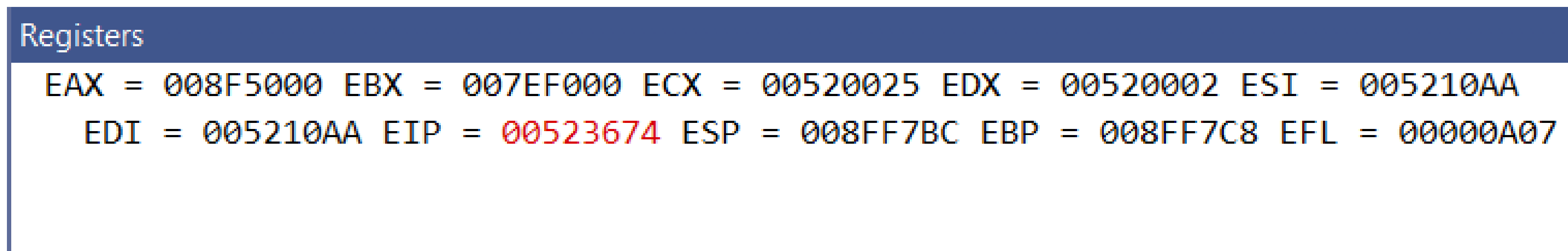
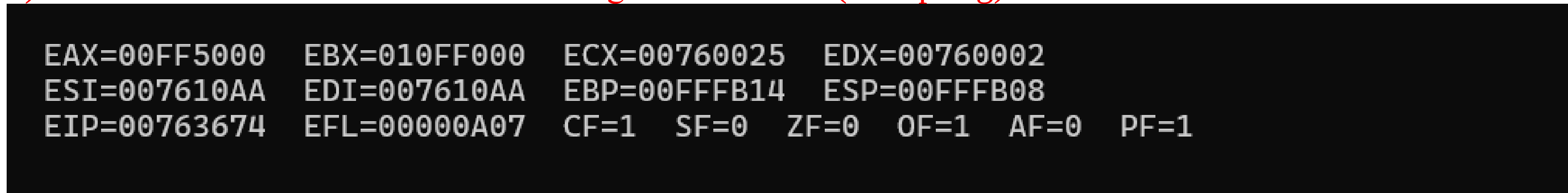
```
MOV DX, 0          ; Clear DX
MOV AX, 1000h       ; Load 1000h into AX
MOV CX, 25h         ; Load 25h into CX
MUL CX              ; Multiply AX by CX, storing the result in DX:AX
```

Answer Q5

- a) Screenshot of full program (.asm) :


```
Lab 2 Question 5 (Global Scope)
1 TITLE Lab 2 Question 5
2 ; Author: Group 1
3 ; Date: 19 May 2024
4
5 include Irvine32.inc
6
7 .code
8
9 main proc
10
11 MOV DX, 0; Clear DX
12 MOV AX, 1000h; Load 1000h into AX
13 MOV CX, 25h; Load 25h into CX
14 MUL CX; Multiply AX by CX, storing the result in DX : AX
15
16 call DumpRegs ; display the registers
17 exit
18
19 main ENDP
20 END main
```

b) Paste here the screenshot of the final registers' content (DumpReg)



Q6. Given the following instructions as is Code Snippet 2.

- a) Write a full program to execute the Code Snippet 2.
- b) What are the contents of the related registers after Code Snippet 2 is executed? Paste the screenshot of DumpReg.

```
; Code Snippet 2 (DIV BL)
MOV DX, 0           ; Clear DX to form the 16-bit dividend in DX:AX
MOV AX, 803h        ; Load the dividend (8003h) into AX
MOV BL, 10h         ; Load the divisor (10h) into BL
DIV BL              ; Divide DX:AX by BL, whereby AX=quotient & DX=remainder
```

Answer Q6

- a) Screenshot of full program (.asm) :


```

Lab 2 q6 (Global Scope)
1  TITLE lab2 Question 6
2  ; Author: Group 1
3  ; Date: 19 May 2023
4
5  include Irvine32.inc
6
7
8  .code
9
10 main proc
11
12     MOV DX, 0; Clear DX to form the 16 - bit dividend in DX : AX
13     MOV AX, 803h; Load the dividend(8003h) into AX
14     MOV BX, 10h; Load the divisor(10h) into BX
15     DIV BX; Divide DX : AX by BX, whereby AX = quotient & DX = remainder
16
17     call DumpRegs; display the register
18                                     ;
19
20     exit ≤ 4ms elapsed
21
22     main ENDP
23
24     END main

```

b) Paste here the screenshot of the final registers' content (DumpReg):

Registers

EAX = 00F60080 EBX = 011F0010 ECX = 00E210AA EDX = 00E20003 ESI = 00E210AA
 EDI = 00E210AA EIP = 00E23674 ESP = 00F6F7D4 EBP = 00F6F7E0 EFL = 00000246

```

EAX=00F60080  EBX=011F0010  ECX=00E210AA  EDX=00E20003
ESI=00E210AA  EDI=00E210AA  EBP=00F6F7E0  ESP=00F6F7D4
EIP=00E23674  EFL=00000246  CF=0   SF=0   ZF=1   OF=0   AF=0   PF=1

```